Expression Parsing and the Shunting-Yard Algorithm

Infix, postfix, and two stacks' worth of patience

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 18 Thursday, November 5, 2026

Today

1. Why $3 + 4 \times 2$ is **11**, not 14: **precedence**.
2. Three ways to write an expression: **infix**, **prefix**, **postfix**. (Last class: the three traversals of an expression tree.)
3. **Evaluating** postfix with one stack.
4. **Converting** infix to postfix: the **shunting-yard** algorithm (a stack of operators that wait).

- **Exercise 8 (Shunting-Yard Trace)** is posted today and is due **Thursday, Nov 12**. Today's trace tables are the model for it.
- Project 4 (Expression Evaluator) builds on today's ideas. It is posted later; this class is the preview.

Question: What Does a Calculator Print?

Evaluate $3 + 4 \times 2$ the way standard arithmetic does (the way a calculator that respects precedence would).

(a) Which answer is correct, 11 or 14?
(b) Reading left to right, what is the *first* operator we meet? Why can we not do it yet?

Question: What Does a Calculator Print?

Evaluate $3 + 4 \times 2$ the way standard arithmetic does (the way a calculator that respects precedence would).

(a) Which answer is correct, 11 or 14?
(b) Reading left to right, what is the *first* operator we meet? Why can we not do it yet?

(a) 11. The rule of precedence: $\times$ and $\div$ bind tighter than $+$ and $-$.

(b) We meet $+$ first, but its right operand is not 4 alone: it is 4×2. The $+$ **has to wait** until we have seen what follows.

Analogy: you read the expression aloud to a friend who does the arithmetic. After "three plus. . ." your friend cannot write anything yet, because what is added to 3 is not known. So they put a **sticky note** "3+" aside, finish $4 \times 2 = 8$, and only then pick the note back up: $3 + 8 = 11$. If they added at once they would get $(3 + 4) \times 2 = 14$. **Waiting** is the whole idea of today.

Three Notations

The operator can go **between**, **before**, or **after** its operands.

Infix (operator between)	**Prefix** (before)	**Postfix** (after)
`3 + 4`	`+ 3 4`	`3 4 +`
`3 + 4 * 2`		
`(3 + 4) * 2`		

Question: write `(3 + 4) * 2` in postfix first, then compare.

Three Notations

The operator can go **between**, **before**, or **after** its operands.

Infix (operator between)	**Prefix** (before)	**Postfix** (after)
`3 + 4`	`+ 3 4`	`3 4 +`
`3 + 4 * 2`	`+ 3 * 4 2`	`3 4 2 * +`
`(3 + 4) * 2`	`* + 3 4 2`	`3 4 + 2 *`

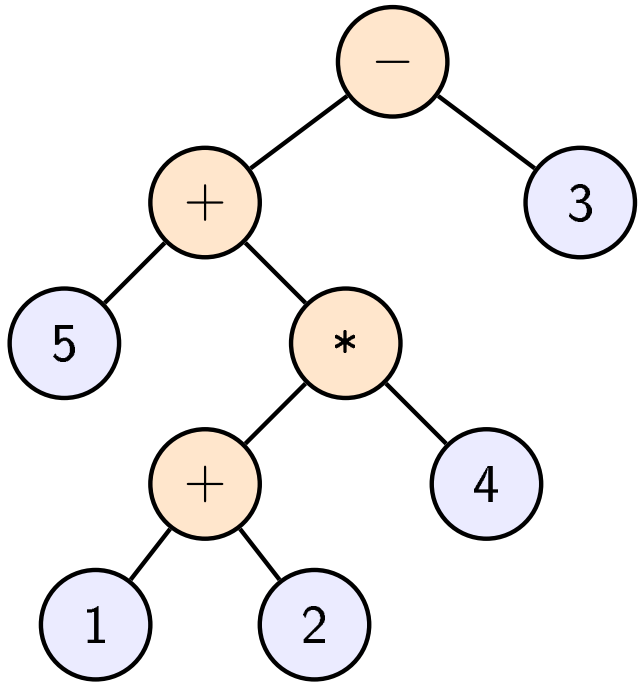
Question: write `(3 + 4) * 2` in postfix first, then compare. Notice that **postfix has no parentheses and no precedence rules**: the order of the tokens already says what to do first. That is why a stack can evaluate it with no precedence decisions to make.

Worked Example: `5 + (1 + 2) * 4 - 3`

Tree. The final tree from the previous slide.

Read. At every node, recursively (left subtree before right):

- **post**(node) = post(left), post(right), node
- **pre**(node) = node, pre(left), pre(right)

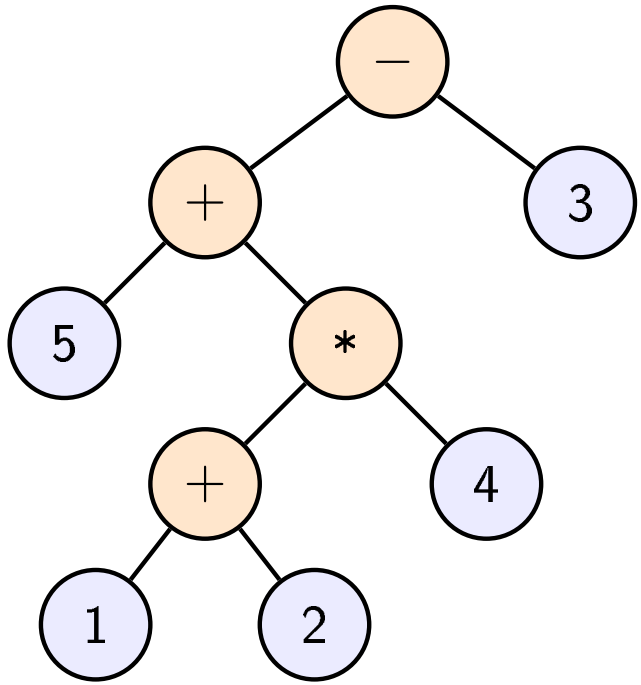


Worked Example: `5 + (1 + 2) * 4 - 3`

Tree. The final tree from the previous slide.

Read. At every node, recursively (left subtree before right):

- **post**(node) = post(left), post(right), node
- **pre**(node) = node, pre(left), pre(right)



Post-order: `5 1 2 + 4 * + 3 -`

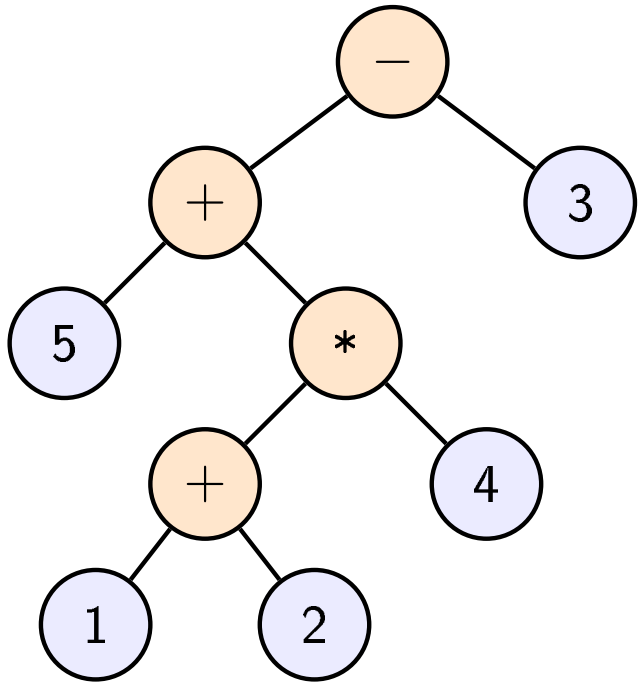
Check: the post-order string is the postfix that the shunting-yard algorithm (a later class) produces.

Worked Example: `5 + (1 + 2) * 4 - 3`

Tree. The final tree from the previous slide.

Read. At every node, recursively (left subtree before right):

- **post**(node) = post(left), post(right), node
- **pre**(node) = node, pre(left), pre(right)



Post-order: `5 1 2 + 4 * + 3 -`

Pre-order: `- + 5 * + 1 2 4 3`

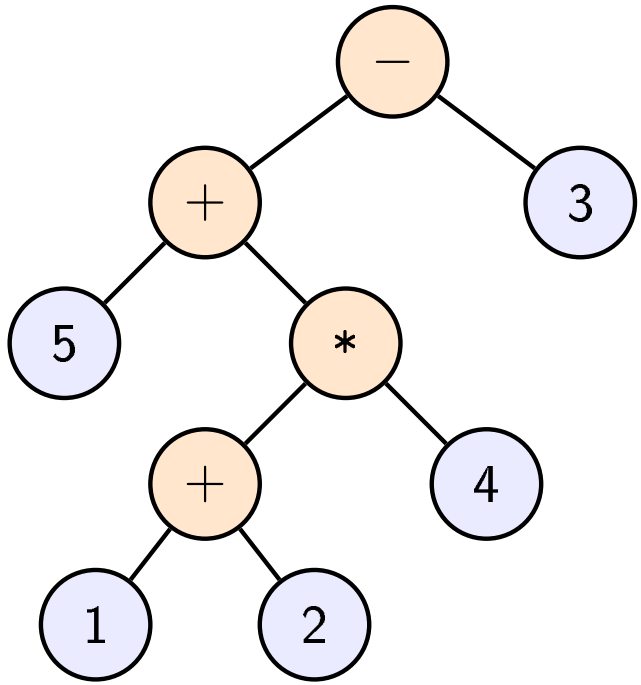
Check: the post-order string is the postfix that the shunting-yard algorithm (a later class) produces.

Worked Example: `5 + (1 + 2) * 4 - 3`

Tree. The final tree from the previous slide.

Read. At every node, recursively (left subtree before right):

- **post**(node) = post(left), post(right), node
- **pre**(node) = node, pre(left), pre(right)



Post-order: `5 1 2 + 4 * + 3 -`

Pre-order: `- + 5 * + 1 2 4 3`

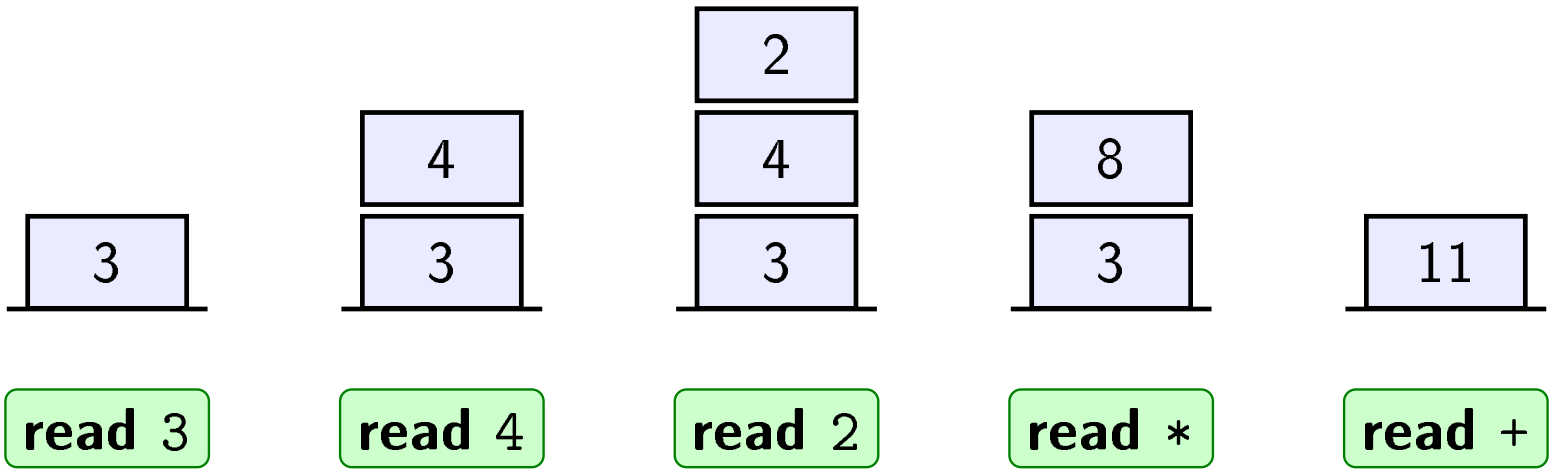
In-order: `5 + 1 + 2 * 4 - 3` (reads as 11)

`((5 + ((1 + 2) * 4)) - 3)` (with parentheses: 14)

Check: the post-order string is the postfix that the shunting-yard algorithm (a later class) produces.

Evaluating Postfix: One Stack

Rule. Read tokens left to right. **Number:** push it. **Operator:** pop two values, apply the operator, push the result.



3 4 2 * +: read *: pop 2, pop 4, push $4 \times 2 = 8$. Read +: pop 8, pop 3, push 11. The answer is the single value left.

Question: Which Operand Comes Off First?

Evaluate the postfix expression `8 2 / 3 -`.

(a) When we read /, the stack holds 8 and 2 (the 2 on top). What are a and b in "a / b"?
(b) What is the final answer?

Question: Which Operand Comes Off First?

Evaluate the postfix expression `8 2 / 3 -`.

(a) When we read /, the stack holds 8 and 2 (the 2 on top). What are a and b in "a / b"?
(b) What is the final answer?

(a) The **first pop is the right operand** $b = 2$, the second pop is the left operand $a = 8$. So $8/2 = 4$, not $2/8$.

(b) Push 4. Push 3. Read -: pop $b = 3$, pop $a = 4$: $4 - 3 = 1$.

For $+$ and $\times$ the order does not matter, so a wrong order **passes the easy tests**. It fails on $-$ and $/$.

Code: Tokenize

Tokens are **strings**, so that 12 is one token, not two.

*Assumes valid input: nonnegative integers, binary + - * /, balanced parentheses (no unary minus or decimals). The last branch does not check characters.*

```
std::vector<std::string> tokenize(const std::string& text) {
    std::vector<std::string> tokens;
    std::size_t i = 0;
    while (i < text.size()) {
        if (std::isspace(static_cast<unsigned char>(text[i]))) { ++i; }
        else if (std::isdigit(static_cast<unsigned char>(text[i]))) {
            std::string num;
            while (i < text.size() &&
                   std::isdigit(static_cast<unsigned char>(text[i]))) {
                num += text[i++];
            }
            tokens.push_back(num);                    // a whole number, e.g. "12"
        } else {
            tokens.push_back(std::string(1, text[i++])); // assumed to be one of
                  + - * / ( )
        }
    }
    return tokens;
}
// tokenize("12+3*(4-1)") gives {"12", "+", "3", "*", "(", "4", "-", "1", ")"}
```

Code: Evaluate a Postfix Queue

`isOp(t)` is true for `+ - * /`. The queue is passed **by value**: the function consumes its own copy.

```
int evalPostfix(std::queue<std::string> postfix) {
    std::stack<int> values;
    while (!postfix.empty()) {
        std::string t = postfix.front();
        postfix.pop();
        if (isOp(t)) {
            int b = values.top(); values.pop();  // right operand first
            int a = values.top(); values.pop();  // then the left operand
            if (t == "+")    { values.push(a + b); }
            else if (t == "-") { values.push(a - b); }
            else if (t == "*") { values.push(a * b); }
            else               { values.push(a / b); }
        } else {
            values.push(std::stoi(t));           // a number: push it
        }
    }
    return values.top();                         // the one value left
}
```

Note: `std::stack<int>` means **integer arithmetic**: `7 / 2` is 3, `-7 / 2` is -3; dividing by zero is not permitted. Assumes a valid postfix sequence of nonnegative integer literals and binary operators, with all literals and intermediate results representable as `int` (an oversized literal makes `std::stoi` throw; overflow is undefined). A complete evaluator must validate.

Converting Infix to Postfix: The Idea

Goal: turn `3 + 4 * 2` into `3 4 2 * +` using a stack, one token at a time.

Question: look at `3 + 4 * 2` and `3 4 2 * +`. (a) Did the **numbers** change order? (b) Which operator was written first, and which came out first?

Converting Infix to Postfix: The Idea

Goal: turn `3 + 4 * 2` into `3 4 2 * +` using a stack, one token at a time.

Question: look at `3 + 4 * 2` and `3 4 2 * +`. (a) Did the **numbers** change order? (b) Which operator was written first, and which came out first?

(a) No: `3, 4, 2` stay in the same order. Numbers can go straight to the output.

(b) $+$ was written first but comes out **last**; $\times$ was written second and comes out first. Operators come out in a **different order**, so they need a place to wait. A **stack** is that place: the newest waiting operator is the first to leave.

Analogy: a railway **shunting yard**, where cars wait on a side track and are released in the order the schedule needs. That is where the algorithm gets its name.

The Shunting-Yard Rules

Two containers: an **output** list and an **operator stack**. For each token:

Token	**Action**
Number	Append to the output.
Operator o	**While** the top of the stack is an operator with **precedence** $\geq$ that of o: pop it to the output. Then push o.
(	Push it.
)	Pop operators to the output until (is on top; discard the (.
End	Pop everything left on the stack to the output.

Precedence: $* \; / = 2$; $+ \; - = 1$. The operators here are **left-associative**: $8 - 3 - 2$ means $(8 - 3) - 2$. That is why we pop on $\geq$ (equal), not just $>$.

Trace 1: 3 + 4 * 2

Fill in the last two columns, then check.

#	Token	Action	Output (left to right)	Stack (bottom → top)
1	3	output		
2	+	push +		
3	4	output		
4	*	push *		
5	2	output		
6	*end*	pop *		
7	*end*	pop +		

Trace 1: 3 + 4 * 2

Fill in the last two columns, then check.

#	Token	Action	Output (left to right)	Stack (bottom → top)
1	3	output	3	
2	+	push +	3	+
3	4	output	3 4	+
4	*	push *	3 4	+ *
5	2	output	3 4 2	+ *
6	*end*	pop *	3 4 2 *	+
7	*end*	pop +	3 4 2 * +	

Row 4: $\times$ has **higher** precedence than the $+$ on top, so nothing is popped. The $+$ keeps waiting. At the end, $\times$ is popped before $+$.

Trace 2: Parentheses, `(3 + 4) * 2`

#	Token	Action	Output (left to right)	Stack (bottom → top)
1	(	push (		
2	3	output		
3	+	push +		
4	4	output		
5	)	pop +; discard (		
6	*	push *		
7	2	output		
8	*end*	pop *		

Trace 2: Parentheses, (3 + 4) * 2

#	Token	Action	Output (left to right)	Stack (bottom → top)
1	(	push (		(
2	3	output	3	(
3	+	push +	3	(+
4	4	output	3 4	(+
5	)	pop +; discard (	3 4 +	
6	*	push *	3 4 +	*
7	2	output	3 4 + 2	*
8	*end*	pop *	3 4 + 2 *	

The (is a wall: nothing pops it except its own). At row 5 the) releases the $+$. Compare with Trace 1: the same numbers, a different output, because the parentheses forced $+$ out **before** $\times$ arrived.

Question: Equal Precedence, 8 - 3 - 2

Infix 8 - 3 - 2 should give $(8 - 3) - 2 = 3$.

(a) Suppose the second - just pushed (no popping on equal precedence). What postfix comes out, and what does it evaluate to?
(b) Now follow the real rule.

#	Token	Action	Output (left to right)	Stack (bottom → top)
1	8	output		
2	-	push -		
3	3	output		
4	-	pop -; push -		
5	2	output		
6	*end*	pop -		

Question: Equal Precedence, 8 - 3 - 2

Infix 8 - 3 - 2 should give $(8 - 3) - 2 = 3$.

(a) Suppose the second - just pushed (no popping on equal precedence). What postfix comes out, and what does it evaluate to?
(b) Now follow the real rule.

#	Token	Action	Output (left to right)	Stack (bottom → top)
1	8	output	8	
2	-	push -	8	-
3	3	output	8 3	-
4	-	pop -; push -	8 3 -	-
5	2	output	8 3 - 2	-
6	*end*	pop -	8 3 - 2 -	

(a) Without the pop: 8 3 2 - -, which evaluates to $8 - (3 - 2) = 7$, wrong. **(b)** Row 4 pops the first - because it has equal precedence: 8 3 - 2 - $= 3$, correct.

Trace 3: Everything at Once, `5 + (1 + 2) * 4 - 3`

#	Token	Action	Output (left to right)	Stack (bottom → top)
1	5			
2	+			
3	(			
4	1			
5	+			
6	2			
7	)			
8	*			
9	4			
10	-			
11	3			
12	*end*			

Your turn: before the actions appear, decide each token's action yourself.

Trace 3: Everything at Once, `5 + (1 + 2) * 4 - 3`

#	Token	Action	Output (left to right)	Stack (bottom → top)
1	5	output		
2	+	push +		
3	(	push (		
4	1	output		
5	+	push +		
6	2	output		
7	)	pop +; discard (		
8	*	push *		
9	4	output		
10	-	pop *; pop +; push -		
11	3	output		
12	*end*	pop -		

Trace 3: Everything at Once, 5 + (1 + 2) * 4 - 3

#	Token	Action	Output (left to right)	Stack (bottom → top)
1	5	output	5	
2	+	push +	5	+
3	(	push (	5	+ (
4	1	output	5 1	+ (
5	+	push +	5 1	+ (+
6	2	output	5 1 2	+ (+
7	)	pop +; discard (	5 1 2 +	+
8	*	push *	5 1 2 +	+ *
9	4	output	5 1 2 + 4	+ *
10	-	pop *; pop +; push -	5 1 2 + 4 * +	-
11	3	output	5 1 2 + 4 * + 3	-
12	*end*	pop -	5 1 2 + 4 * + 3 -	

Row 10 is the busiest: the new - pops * (higher), then + (equal), then stops because the stack is empty. Result 5 1 2 + 4 * + 3 - = 14 (evaluate it to check).

Code: Shunting-Yard

```
int prec(const std::string& op) {
    if (op == "*" || op == "/") { return 2; }
    return 1;                                          // "+" and "-"
}

std::queue<std::string> shuntingYard(const std::vector<std::string>& tokens) {
    std::queue<std::string> output;
    std::stack<std::string> ops;
    for (const std::string& t : tokens) {
        if (isOp(t)) {
            while (!ops.empty() && isOp(ops.top()) && prec(ops.top()) >= prec(t)) {
                output.push(ops.top()); ops.pop();   // stronger or equal leaves
            }
            ops.push(t);
        } else if (t == "(") {
            ops.push(t);
        } else if (t == ")") {
            while (ops.top() != "(") { output.push(ops.top()); ops.pop(); }
            ops.pop();                                 // discard the "("
        } else {
            output.push(t);                            // a number goes straight out
        }
    }
    while (!ops.empty()) { output.push(ops.top()); ops.pop(); }
    return output;
}
```

Only the operator `while` loop encodes precedence; it stops at a `(`. **Valid-input version; validation omitted** (see Common Mistakes).

Common Mistakes

- **Operands popped in the wrong order** when evaluating: the **first** pop is the **right** operand.
- **Popping only on** $>$: $8 - 3 - 2$ becomes 7. Use $\geq$ for left-associative operators.
- **Forgetting the final flush**: after the last token, the stack still holds operators. They belong in the output.
- **Letting an operator pop a** `(`: the `(` must stay until its `)`. Check `isOp(ops.top())`.
- **No validation** (valid input assumed; Project 4 must add it): a `)` with no `(` (empty stack; `top()` is undefined) or a leftover `(`; fewer than two values for a binary operator, or not exactly one value at the end; invalid tokens; infix operand/operator order (infix `1 2 +` would slip through).
- **Tokenizing by single characters**: `12` must stay one token.

The complete, runnable code (with headers and `isOp`) is in `class17_expression_evaluator.cpp`.

Practice

1. Convert `2 * 3 + 4` to postfix.
2. Convert `9 - 4 - 3` to postfix and evaluate it.
3. Evaluate the postfix expression `5 1 2 + 4 * + 3 -`.
4. What is the largest number of values on the evaluation stack while evaluating `3 4 2 * +`?
5. Draw the expression tree of `2 * 3 + 4` and read it in post-order.
6. Tokenize, convert to postfix, and evaluate `12 / (2 + 1) - 1`.

Practice

1. Convert `2 * 3 + 4` to postfix.
2. Convert `9 - 4 - 3` to postfix and evaluate it.
3. Evaluate the postfix expression `5 1 2 + 4 * + 3 -`.
4. What is the largest number of values on the evaluation stack while evaluating `3 4 2 * +`?
5. Draw the expression tree of `2 * 3 + 4` and read it in post-order.
6. Tokenize, convert to postfix, and evaluate `12 / (2 + 1) - 1`.

1. `2 3 * 4 +`. (When $+$ arrives, the $*$ on top has higher precedence, so $*$ leaves first.)
2. `9 4 - 3 -` $= (9 - 4) - 3 = 2$.
3. $5 + (1 + 2) \times 4 - 3 = 14$.
4. 3 (the values $3, 4, 2$ are on the stack together).
5. Root $+$ with left child $\times(2, 3)$ and right child 4. Post-order: `2 3 * 4 +`.

Practice

1. Convert `2 * 3 + 4` to postfix.
2. Convert `9 - 4 - 3` to postfix and evaluate it.
3. Evaluate the postfix expression `5 1 2 + 4 * + 3 -`.
4. What is the largest number of values on the evaluation stack while evaluating `3 4 2 * +`?
5. Draw the expression tree of `2 * 3 + 4` and read it in post-order.
6. Tokenize, convert to postfix, and evaluate `12 / (2 + 1) - 1`.

6. Tokens: `12 / ( 2 + 1 ) - 1`.
Postfix: `12 2 1 + / 1 -`.
Value: $12/(2+1)-1=4-1=3$. The 12 stays one token, and the parentheses make $+$ go before $/$.

Summary and Preview

- **Postfix** needs no parentheses or precedence: one stack evaluates it.
- **Shunting-yard**: numbers go straight out; operators **wait on a stack**, and leave when precedence and associativity require it, at a closing parenthesis, or in the final flush. For n tokens, each operator is pushed and popped once: $O(n)$ time and $O(n)$ auxiliary space.
- **Parentheses** are walls on the stack; precedence is the pop rule.
- An **expression tree** shows the structure; its traversals are the three notations.

Coming up

Nov 10: searching: linear search, binary search, and why sorted data matters.
Nov 12: Exercise 8 (Shunting-Yard Trace) due.
Later: Project 4 (Expression Evaluator) puts tokenizing, conversion, and evaluation together.